LILA BLACK

Player Journey Visualizer

Live Tool	https://lila-tool.vercel.app/
GitHub Repo	https://github.com/pratiksingh-dev/lila-tool

What I Built

> **The core principle**
> A level designer should open a URL and see useful data immediately — no setup, no engineer, no configuration. Every decision I made traces back to this.

I built a static web tool that turns LILA BLACK's raw parquet telemetry into an interactive map explorer. The full dataset — 89,000 events across 796 matches on 3 maps — loads in under 2 seconds and renders at 60fps with no server, no database, and no build step.

Features

M	**Real minimap**	All 3 maps with exact coordinate projection from README formula. Validated: 100% of events land on map terrain.
H	**Human vs bot**	Blue paths/dots = humans, pink = bots. Detected via user_id pattern. Toggle independently.
E	**8 event types**	Kill, Killed, BotKill, BotKilled, Loot, KilledByStorm — distinct shapes and colors on canvas.
F	**Filter controls**	Map, date, and match dropdowns. Match list shows K/D/Loot stats so designers pick without guessing.
T	**Timeline playback**	Scrub or play through a match. Works on single match and aggregate view. Slow / Normal / Fast / Ultra.
X	**Heatmap overlays**	Four modes: all movement, kill zones, death zones, loot hotspots. Powered by heatmap.js via CDN.
G	**Zone coverage grid**	8×8 overlay — blue = visited, red = zero traffic. Shows which map areas players never reach.
I	**Live Insights panel**	Right panel updates in real time: kill concentration, dead zones, bot vs human behavior, loot-fight distance.

My Process

1 Understand the data first

Before writing any frontend code I ran exploratory analysis in Python. I found: the event column is stored as bytes and needs decoding; bot detection works via user_id regex; the coordinate formula is exact and validates to 100% on-map; one file is 0 bytes and crashes the pipeline if not handled.

2 Decide before building

The user is a level designer, not a data scientist. They need answers in 10 seconds. So: aggregate heatmap as the default view (useful immediately), match dropdown shows stats upfront, insights surface automatically rather than waiting for the designer to calculate them.

3 Pipeline first, frontend second

One Python script converts all parquet files to clean JSON — decoding, bot detection, coordinate normalization, per-map splitting. The frontend only fetches pre-processed data. This separation means updating with new match data requires running one script, not touching the frontend.

4 Ship something that actually works

Vanilla HTML/JS with no framework means no build step, no dependency drift, and no setup for whoever opens it. Canvas instead of SVG handles 89k events without frame drops. Vercel auto-deploys on every push. The tool is live, fast, and requires nothing from the evaluator to run.

Three Insights from the Data

#	Insight	The data	Actionable recommendation
1	**Kill funnel on Ambrose Valley**	52% of all kills happen in one 8×8 grid zone. Loot centroid is 43 world units from kill centroid — players loot then travel south to fight.	Cross-reference kill cluster coords (~−6, +13) with level doc. If unintentional: open alternate route. Target: no single zone above 30% of kills.
2	**Lockdown's south half is a dead zone**	97% of position events are in the top half. Gas Station and Engineer's Quarters have 0 kills, 0 loot, 0 visits across all 4 matches.	Check storm direction. Add loot or extraction points in the south. Metric: zone visit rate above 60% for all named zones. Current rate: ~0%.
3	**Bots are not reliable design proxies**	Bot death rate is 28× higher than humans. Deaths cluster in a 50-unit radius vs. humans who spread across 4 quadrants. High centroid gap confirmed.	Separate bot and human data in all design analysis. Investigate geometry at (~10, −30) on AV. Metric: spatial divergence score per map.

Tech at a Glance

Stack

- Python · pandas · pyarrow (pipeline)
- Vanilla HTML/JS (frontend)
- HTML5 Canvas API (rendering)
- heatmap.js v2.0.5 via CDN (density maps)
- Vercel (hosting, auto-deploy on push)

Repository structure

- public/index.html — entire frontend
- public/data/*.json — pre-processed events

Key decisions

- Static JSON over live server — no runtime cost, sub-200ms loads
- Canvas over SVG — handles 89k events without DOM freeze
- Per-map lazy loading — only fetch the map selected
- Vanilla JS over React — no build step, opens anywhere

- public/minimaps/*.jpg — map images
- scripts/process_data.py — ETL pipeline
- ARCHITECTURE.md, INSIGHTS.md, README.md

If I had more time

- Named zone overlays with per-zone click stats
- Day-over-day trend chart (Feb 10–14 shift)
- Bot divergence score — automated fidelity flag
- Bundle heatmap.js locally — remove CDN dependency
- GitHub Action to auto-run pipeline on new data